

Chương 19: Huấn luyện và Triển khai Mô hình TensorFlow Quy mô Lớn

Giảng viên

Đại học Đông Á

August 11, 2026

Mục tiêu của Chương

- Đưa mô hình ML vào môi trường thực tế (Production) với **TensorFlow Serving** và **Vertex AI**.
- Đưa AI lên Thiết bị di động (Mobile) và thiết bị nhúng với **TFLite**.
- Chạy AI trực tiếp trên trình duyệt bằng **TensorFlow.js**.
- Tối ưu hóa hiệu năng tính toán với **GPU / TPU**.
- Phân tán khối lượng huấn luyện khổng lồ trên hàng nghìn máy chủ với **Distribution Strategies API**.

Mục lục

1. Phục vụ một mô hình TensorFlow (TF Serving)
2. Triển khai AI lên Thiết bị Di động & Web
3. Tăng tốc bằng GPU/TPU và Phân tán Tính Toán
4. Kiến trúc Đồng bộ và Bất đồng bộ
5. API Chiến lược Phân phối & Huấn luyện Cụm
6. Tổng kết

1.1 Vòng đời dự án Machine Learning (MLOps)

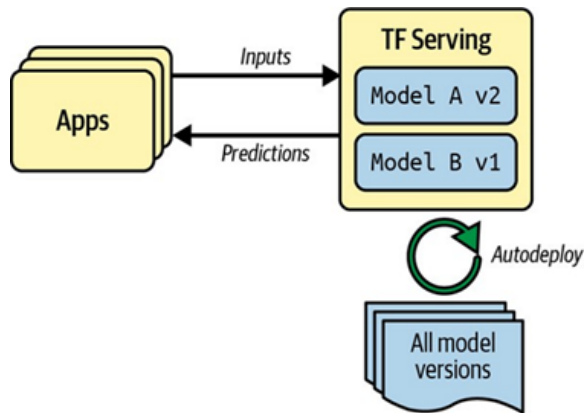
- Có được một mô hình học máy đạt độ chính xác cao mới chỉ là **bước khởi đầu**.
- Một kỹ sư ML thực thụ phải biết cách đưa mô hình đó ra phục vụ hàng triệu người dùng (Deployment).
- **MLOps (Machine Learning Operations)**: Tự động hóa quá trình huấn luyện, triển khai, theo dõi và cập nhật mô hình một cách mượt mà.

1.2 Những thách thức khi đưa AI vào Thực tế

- **Khả năng mở rộng (Scalability):** Hệ thống phải chịu tải được hàng ngàn request mỗi giây mà không bị sập.
- **Độ trễ (Latency):** Người dùng yêu cầu nhận được dự đoán (VD: Dịch thuật, Nhận diện) ngay lập tức.
- **Quản lý phiên bản (Versioning):** Phải liên tục nạp mô hình mới (chính xác hơn) lên server mà không làm gián đoạn dịch vụ cũ (Zero Downtime).

1.3 Giới thiệu TensorFlow Serving

- **TensorFlow Serving (TFS):** Công cụ bằng C++ hiệu năng cực cao của Google, chuyên dùng để chạy các mô hình ML trong Production.
- Thay vì dùng Flask hay Django (rất chậm vì dùng Python), TFS tối ưu hóa khả năng tận dụng GPU và CPU đa luồng.



Hình 19-1: Hệ sinh thái TensorFlow Extended (TFX) & TF Serving

1.4 Kiến trúc của TensorFlow Serving

- TFS liên tục theo dõi một thư mục mô hình.
- Bất cứ khi nào bạn thả một thư mục phiên bản mới (VD: v2) vào, TFS sẽ tự động tải nó lên bộ nhớ.
- Request mới sẽ được chuyển hướng sang v2, trong khi các request cũ vẫn chạy trên v1 cho đến khi xong hẳn, rồi v1 mới bị xóa.

1.5 Xuất mô hình (Exporting Model)

- Để dùng được TFS, ta phải lưu mô hình dưới định dạng **SavedModel**.
- Thư mục SavedModel chứa: file saved_model.pb (kiến trúc mạng) và thư mục variables (trọng số).

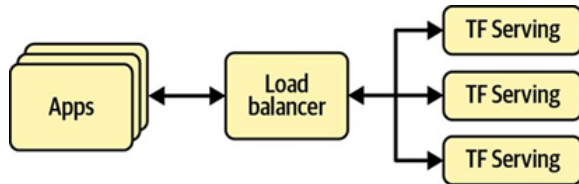
```
import tensorflow as tf

model = ... # Huan luyen mo hinh xong
model_version = "0001"
model_name = "my_mnist_model"
model_path = os.path.join(model_name, model_version)

# Luu thanh dinh dang SavedModel
tf.saved_model.save(model, model_path)
```

1.6 REST API và gRPC

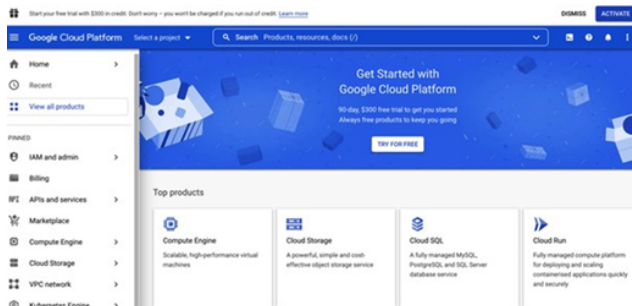
- TFS hỗ trợ 2 giao thức mạng:
- **REST API:** Dùng JSON qua HTTP. Dễ cấu hình, dễ debug, thích hợp cho Ứng dụng Web. (Chậm hơn vì phải parse JSON).
- **gRPC:** Giao thức nhị phân của Google. Cực kỳ nhanh, băng thông thấp. Phù hợp cho giao tiếp Server-to-Server.



Hình 19-2: REST vs gRPC trong giao tiếp
Microservices

1.7 Tạo dịch vụ Dự đoán trên Vertex AI (Google Cloud)

- Bạn không cần tự thuê Server và cài Docker TFS!
- **Vertex AI** là nền tảng quản lý MLOps toàn diện của Google Cloud.
- Bạn chỉ cần upload SavedModel lên Cloud Storage, nhấn Deploy, Google sẽ tự động tạo Endpoint (API) cho bạn.



Hình 19-3: Quy trình đưa Mô hình lên Vertex AI

1.8 Mã nguồn Triển khai trên Vertex AI

```
from google.cloud import aiplatform

# 1. Tai mo hinh tu Google Cloud Storage vao Vertex AI Model Registry
model = aiplatform.Model.upload(
    display_name="my_mnist_model",
    artifact_uri="gs://my-bucket/my_mnist_model/0001",
    serving_container_image_uri="us-docker.pkg.dev/vertex-ai/prediction/tf2-cpu.2-8:
        latest"
)

# 2. Trien khai mo hinh len mot Endpoint
endpoint = model.deploy(
    machine_type="n1-standard-4",
    min_replica_count=1,
    max_replica_count=5 # Tu dong scale len 5 may chu neu tai cao
)
```

1.9 Dự đoán Hàng loạt (Batch Predictions)

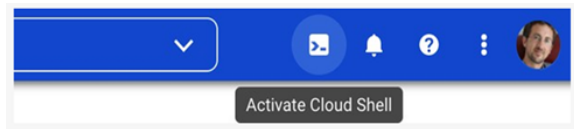
- Endpoint ở trên dùng cho **Online Prediction** (Cần trả lời ngay lập tức từng ảnh một).
- Nếu ta có 1 triệu bức ảnh trong Cloud Storage cần phân loại (không vội), ta sẽ dùng **Batch Prediction**.
- Vertex AI sẽ tự động mượn hàng trăm máy tính, chạy đồng loạt mô hình, ghi kết quả ra file, rồi tắt máy tính để tiết kiệm tiền.

1.10 Tóm tắt: Tự quản lý vs Dùng Cloud (Vertex AI)

- **Tự cài TF Serving (On-premise):** Rẻ hơn, kiểm soát hoàn toàn dữ liệu. Nhưng tốn công bảo trì, cấu hình Load Balancer, Kubernetes phức tạp.
- **Dùng Vertex AI:** Tốn tiền dịch vụ Cloud, nhưng Google tự lo phần Scaling, Load Balancing, và Bảo mật. Giúp kỹ sư ML tập trung vào Mô hình.

2.1 Edge AI - Trí tuệ Nhân tạo tại biên

- Tại sao phải đưa AI xuống chạy trực tiếp trên Điện thoại / Smartwatch?
- **Độ trễ thấp nhất:** Không cần gửi ảnh lên server.
- **Hoạt động Offline:** Vẫn dùng được khi rớt mạng.
- **Bảo mật:** Dữ liệu cá nhân (Khuôn mặt, Giọng nói) không bao giờ rời khỏi thiết bị.



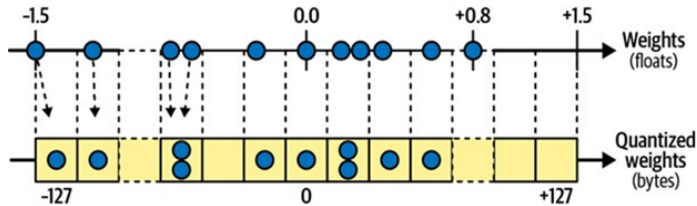
Hình 19-4: Ứng dụng Edge AI trong đời sống

2.2 Giới thiệu TensorFlow Lite (TFLite)

- TFLite là một phiên bản "thu nhỏ" của TensorFlow, được thiết kế đặc biệt cho Điện thoại (Android, iOS) và Vi điều khiển (Arduino, ESP32).
- Dung lượng file thư viện TFLite chỉ tồn vài Trăm Kilobyte!
- Nó không có các hàm huấn luyện mạng (Train), chỉ chứa các hàm chạy dự đoán (Inference).

2.3 Quy trình chuyển đổi TFLite

- **Bước 1:** Train mô hình TF bình thường bằng Python.
- **Bước 2:** Dùng TFLite Converter để nén mô hình thành file .tflite.
- **Bước 3:** Nạp file .tflite vào app Android bằng Java/Kotlin và chạy Interpreter.



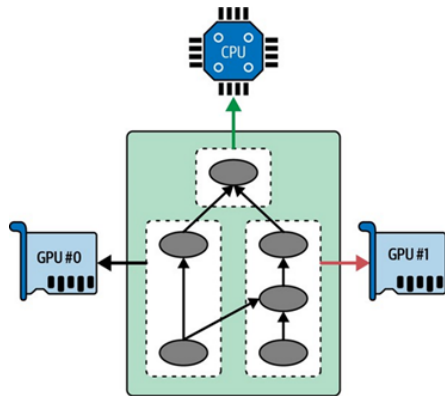
Hình 19-5: Luồng công việc TFLite: Train, Convert, Deploy

2.4 Tối ưu hóa: Lượng tử hóa (Quantization)

- Trọng số nơ-ron mặc định lưu dưới dạng số thực 32-bit (Float32). Rất nặng!
- Lượng tử hóa (Quantization) chuyển trọng số thành số nguyên 8-bit (Int8).
- Lợi ích: **Giảm 4 lần dung lượng** mô hình. Tốc độ tính toán nhanh hơn.
- Cái giá: Giảm độ chính xác một chút xíu (khoảng 1-2%).

2.5 Tối ưu hóa: Cắt tỉa mạng (Pruning)

- **Pruning:** Quá trình tìm và cắt đứt các kết nối (trọng số) có giá trị gần bằng 0 trong mạng nơ-ron.
- Mạng trở nên thưa thớt (Sparse). File nén .zip cực kỳ nhỏ bé.
- **Clustering:** Gom nhóm các trọng số giống nhau lại dùng chung 1 giá trị.



Hình

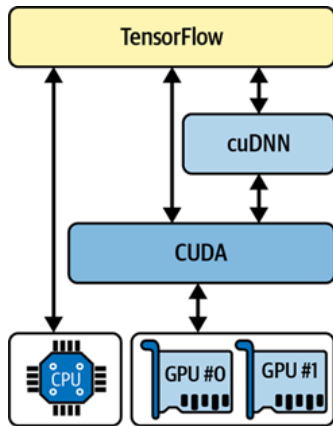
19-6: Kỹ thuật cắt tỉa làm mỏng mạng nơ-ron (Pruning)

2.6 Delegate: Khai thác phần cứng Mobile

- Chạy AI trên CPU điện thoại sẽ làm nóng máy và tốn pin nhanh.
- TFLite hỗ trợ các **Delegate** để chuyển tính toán sang GPU điện thoại hoặc NPU (Neural Processing Unit).
- Ví dụ: Android Neural Networks API (NNAPI) hoặc Apple Core ML.

2.7 TensorFlow.js (TF.js)

- Đưa AI trực tiếp lên Trình duyệt Web (Chrome, Firefox)!
- Viết hoàn toàn bằng JavaScript/TypeScript.
- Khách truy cập web không cần cài đặt bất cứ thư viện nào, AI tự chạy phía Client!



Hình 19-7: Hệ sinh thái TensorFlow.js trên Browser và Node.js

2.8 Đặc điểm nổi bật của TF.js

- **Tận dụng WebGL / WebGPU:** TF.js tự động chuyển các phép nhân ma trận thành mã shader để chạy trên Card Đồ Họa của người dùng duyệt web.
- Tốc độ có thể đạt gần bằng chạy native (C++).
- Rất tuyệt vời cho các demo tương tác AI (ví dụ: Google Teachable Machine).

2.9 Chuyển đổi mô hình cho Web

- Để chạy được trên JS, ta phải chuyển SavedModel sang định dạng Web (model.json và file shard nhị phân).

```
# Cài đặt tfjs converter
pip install tensorflowjs

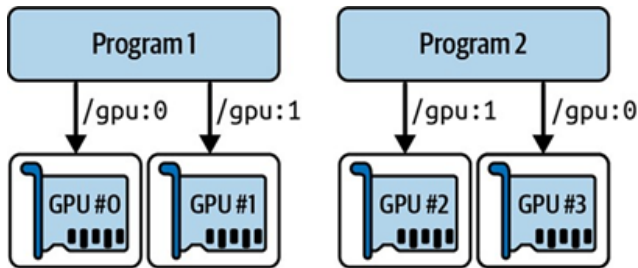
# Lệnh chuyển đổi
tensorflowjs_converter \
    --input_format=keras \
    my_model.h5 \
    my_tfjs_model_folder
```

3.1 Nhu cầu Phần cứng Chuyên dụng

- Huấn luyện mô hình LLM (như GPT-4) hay CNN hàng tỷ tham số tốn cực nhiều phép nhân ma trận liên tiếp.
- CPU mất hàng năm trời để hoàn thành. Ta bắt buộc phải dùng **Phần cứng gia tốc (Accelerators)** như GPU hoặc TPU.

3.2 CPU so với GPU và TPU

- **CPU (Central Processing Unit):** Một vị giáo sư thông thái. Xử lý cực tốt logic phức tạp, nhưng chỉ có vài nhân (cores).
- **GPU (Graphics Processing Unit):** Hàng ngàn học sinh tiểu học. Chỉ biết làm toán cộng nhân đơn giản, nhưng làm đồng thời. Rất hợp cho Ma trận.
- **TPU (Tensor Processing Unit):** Chip do chính Google làm riêng cho phép nhân Ma trận.



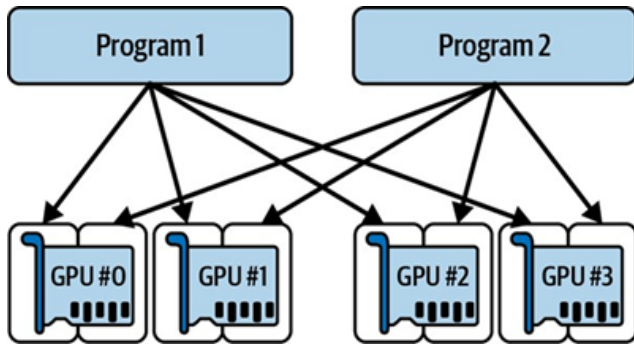
Hình 19-8: CPU (ít ALU phức tạp) vs GPU
(hàng ngàn ALU đơn giản)

3.3 Làm sao có GPU để huấn luyện?

- **Mua máy tính riêng (Local):** Đầu tư Card NVIDIA (RTX 4090). Cần cài CUDA Toolkit và cuDNN. Mất nhiều thời gian bảo trì.
- **Thuê Cloud VM:** AWS EC2 P3/P4, Google Cloud Compute. Chi phí tính theo giờ.
- **Google Colab / Kaggle:** Nền tảng Notebook trực tuyến cung cấp GPU/TPU miễn phí cho sinh viên và cá nhân. Tốt nhất để học!

3.4 Quản lý RAM GPU trong TensorFlow

- Theo mặc định, khi vừa bật lên, TensorFlow sẽ **chiếm toàn bộ VRAM** của GPU để tránh phân mảnh bộ nhớ.
- Nếu bạn muốn chạy 2 mô hình cùng lúc, mô hình 2 sẽ báo lỗi "Out of Memory" (OOM).
- Giải pháp: Bật tính năng `memory_growth` (chỉ cấp phát bộ nhớ khi cần).



Hình 19-9: Vấn đề tràn RAM GPU và cách khắc phục

3.5 Bật chế độ Memory Growth (Code)

```
import tensorflow as tf

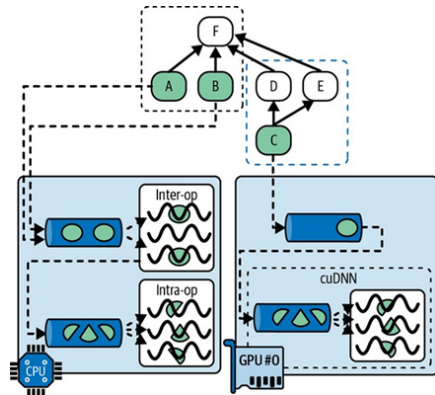
physical_devices = tf.config.list_physical_devices('GPU')
if physical_devices:
    try:
        # Bat che do "can den dau cap den do"
        for gpu in physical_devices:
            tf.config.experimental.set_memory_growth(gpu, True)
    except RuntimeError as e:
        print(e)
```

3.6 Huấn luyện trên Nhiều thiết bị (Multi-Device)

- Khi 1 Card GPU (VD: VRAM 24GB) không chứa nổi Batch Size lớn, ta phải ghép 2, 4 hoặc 8 Card GPU lại trên cùng một bo mạch chủ (Multi-GPU).
- Nếu 8 Card vẫn không đủ, ta ghép 100 máy chủ lại với nhau (Mỗi máy có 8 Card) thông qua mạng tốc độ cao (Multi-Worker).
- Đây là cách mà siêu máy tính huấn luyện ChatGPT (Hàng chục ngàn GPU A100).

3.7 Cách TensorFlow chia tải: Placement

- TensorFlow biểu diễn mọi tính toán bằng **Đồ thị (Graph)**.
- Một thuật toán tên là **Placer** sẽ quyết định nút nào (Node) chạy trên CPU, nút nào chạy trên GPU 0, GPU 1.
- Ta có thể cưỡng ép thủ công: with `tf.device('/GPU:1')`:

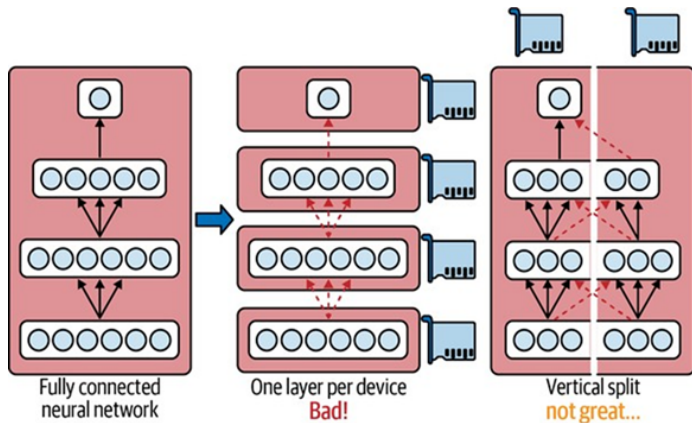


Hình

19-10: Đồ thị tính toán phân bổ trên các thiết bị

3.8 Mô hình Song song (Model Parallelism)

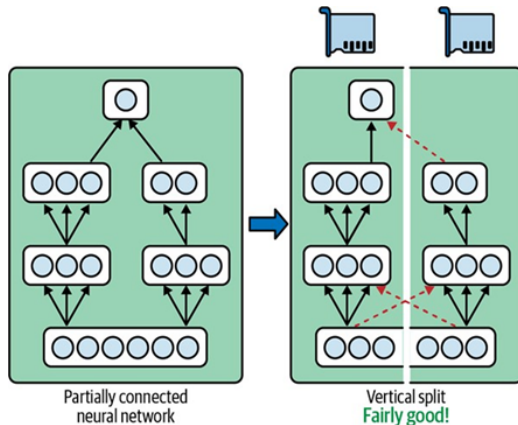
- Nếu bản thân mạng Nơ-ron lớn tới mức 1 GPU không chứa nổi trọng số của nó.
- Giải pháp: Cắt mạng làm đôi. Lớp 1-50 nạp vào GPU 0. Lớp 51-100 nạp vào GPU 1.
- **Nhược điểm:** GPU 1 sẽ phải "ngồi chơi xơi nước" chờ GPU 0 tính xong lớp 50 mới có dữ liệu làm tiếp. Không hiệu quả lắm!



Hình 19-11: Cắt ngang kiến trúc bằng Model Parallelism

3.9 Song song Dữ liệu (Data Parallelism)

- Là cách làm phổ biến nhất.
- **Copy** y hệ mạng nơ-ron lên tất cả các GPU.
- Thay vì nạp batch size = 32 vào 1 GPU. Ta nạp 8 ảnh vào GPU 0, 8 ảnh vào GPU 1, v.v.
- Chúng tính toán song song Gradient, sau đó **cộng gộp Gradient** lại rồi cập nhật cho tất cả.

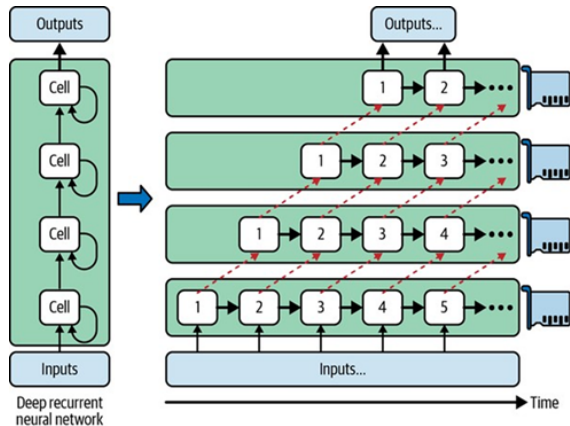


Hình 19-12:

Thuật toán Data Parallelism cơ bản

4.1 Cập nhật Đồng bộ (Synchronous Update)

- Tất cả GPU tính xong Gradient của lô ảnh của mình.
- Mọi người DỪNG LẠI, gửi Gradient cho nhau (Thuật toán **All-Reduce**).
- Trung bình cộng Gradient được tính ra, rồi tất cả cùng tiến thêm 1 bước.
- Điểm nghẽn: GPU nhanh phải đợi GPU chậm (Straggler).

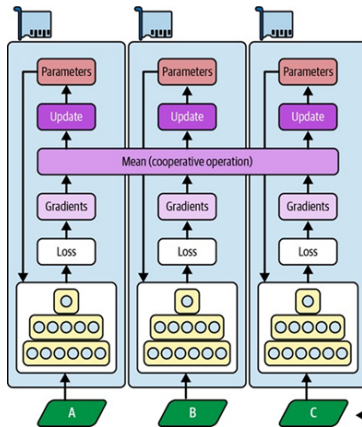


Hình

19-13: Cập nhật Gradient Đồng bộ bằng All-Reduce

4.2 Cập nhật Bất đồng bộ (Asynchronous Update)

- Có một **Parameter Server** (Máy chủ thông số) đứng giữa giữ trọng số thật.
- GPU 0 tính xong Gradient, quăng lên Server, Server cập nhật. GPU 0 lấy trọng số mới về làm luôn.
- Không ai đợi ai.
- Điểm trừ: Gradient bị "cũ" (Stale Gradient) khiến việc đào tạo có thể hỗn loạn (Hội tụ khó hơn).



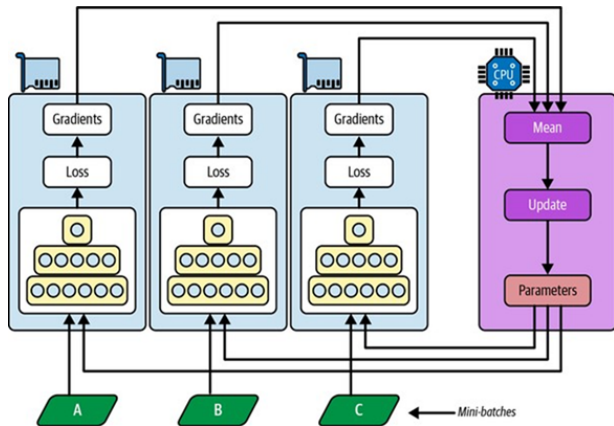
Hình 19-14: Kiến trúc Bất đồng bộ với Parameter Server

5.1 API `tf.distribute.Strategy`

- Việc tự code logic chia dữ liệu và All-Reduce rất cực khổ.
- TensorFlow cung cấp **Distribution Strategies API**. Bạn chỉ cần viết code mô hình Keras 1 lần, TF sẽ tự lo việc phân tán nó đi đâu.
- Khả năng mở rộng từ 1 GPU lên cụm TPU Pods với thay đổi code tối thiểu!

5.2 MirroredStrategy (Đa GPU 1 máy tính)

- Dùng cho một máy chủ (VD: Workstation có 4 Card RTX 4090).
- Nó tự tạo các bản sao (Replicas) trên mỗi GPU.
- Sử dụng **Đồng bộ Data Parallelism** (All-Reduce qua kết nối NVLink cực nhanh).



Hình 19-15: Cơ chế của MirroredStrategy trên 1 Node

5.3 Triển khai MirroredStrategy bằng Keras

```
import tensorflow as tf

# 1. Khai bao chien luoc
strategy = tf.distribute.MirroredStrategy()

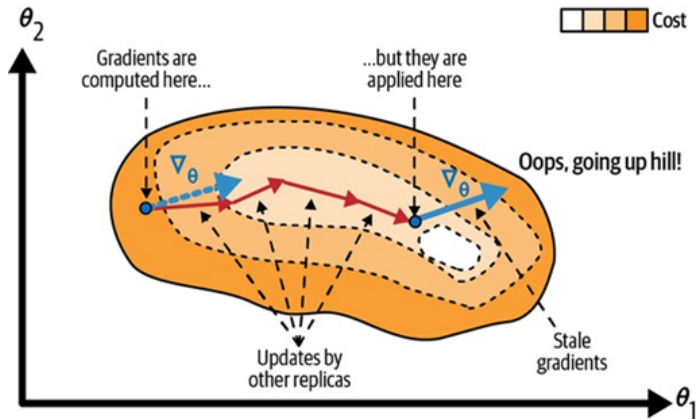
print(f"So luong thiet bi GPU: {strategy.num_replicas_in_sync}")

# 2. Xay dung va bien dich mo hinh BEN TRONG block scope
with strategy.scope():
    model = keras.models.Sequential([
        keras.layers.Dense(128, activation='relu'),
        keras.layers.Dense(10, activation='softmax')
    ])
    model.compile(loss="sparse_categorical_crossentropy", optimizer="sgd")

# 3. Huan luyen binh thuong, TF se tu chia viec!
model.fit(train_dataset, epochs=10)
```

5.4 MultiWorkerMirroredStrategy (Đa máy tính)

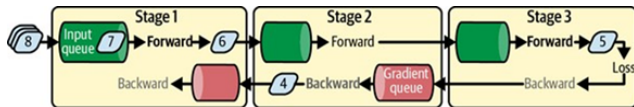
- Khi có 5 máy tính kết nối mạng, mỗi máy có 4 GPU (Tổng 20 GPU).
- Thuật toán tương tự MirroredStrategy nhưng phải liên lạc All-Reduce thông qua mạng LAN/Internet (gRPC).
- Yêu cầu cấu hình biến môi trường TF_CONFIG để các máy biết địa chỉ IP của nhau.



Hình 19-16: Phân tán đào tạo trên một Cụm máy (Cluster)

5.5 Huấn luyện phân tán trên Vertex AI

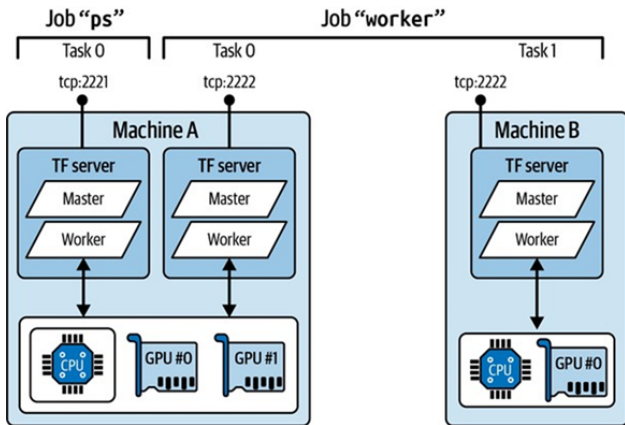
- Tự quản lý cụm 50 máy tính là ác mộng (Lỗi ổ cứng, sập mạng, quá nhiệt).
- Đẩy code lên **Vertex AI Custom Training**.
- Google sẽ khởi động 50 cái máy (Compute Engine) kết nối sẵn, cài sẵn TF, chạy code của bạn. Chạy xong tự động tắt máy.



Hình 19-17: Đóng gói Code bằng Docker để đưa lên Cloud

5.6 Điều chỉnh Siêu tham số (Hyperparameter Tuning)

- Làm sao để biết Learning Rate = 0.01 hay 0.001 là tốt nhất? Batch Size = 32 hay 64?
- Vertex AI cung cấp dịch vụ Vizier. Thuật toán Tối ưu hóa Bayesian sẽ đoán thử các cặp thông số và chạy thử nghiệm (Trial).
- Giúp tìm ra kiến trúc tốt nhất mà không cần thử tay mù quá.



Hình 19-18: Quá trình tìm kiếm siêu tham số qua các Trial

Tổng Kết Chương 19

- Để đưa AI phục vụ hàng triệu người dùng, ta dùng **TF Serving**.
- Đưa AI xuống điện thoại/web để có độ trễ siêu thấp bằng **TFLite** & **TF.js**.
- Gia tốc huấn luyện với **GPU** / **TPU** giúp giảm thời gian từ hàng năm xuống còn vài ngày.
- API **Distribution Strategies** giúp code linh hoạt mở rộng từ 1 máy lên quy mô Cluster đám mây.
- Hệ sinh thái TensorFlow cung cấp bộ công cụ E2E (End-to-End) hoàn hảo cho mọi quy mô doanh nghiệp.

HỎI & ĐÁP